

	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JDBC	VERSIÓN: 1.3 VIGENCIA: 11-05-2020

JDBC

Un "gestor" (o un "motor") de bases de datos es un programa integrado que permite crear, controlar, modificar y acceder a una base de datos, normalmente a través de un lenguaje especial llamado SQL (por "Structured Query Language", o Lenguaje Estructurado de Consulta). Cada gestor de bases de datos se comunica con la base de datos a través de ciertas reglas y procedimientos propios, llamados "protocolos" y manejados por drivers provistos por cada fabricante. Además, cada motor usa una variante diferente de SQL por lo que un programador debe conocer específicamente las diferencias de cada plataforma.

Puesto que los drivers de acceso son diferentes para cada motor, un programa podrá acceder a la base de datos sólo si integra de alguna forma los drivers del motor con el cual esa base se haya creado. Algunos motores muy conocidos son Oracle, SQL Server, MySQL, PostgreSQL, etc.

Para que los programas que usan bases de datos (que en el mundo del desarrollo de software de gestión, son la mayoría) no tengan problemas de incompatibilidad con el motor, se usan administradores de protocolos que permiten agrupar y encapsular los drivers de todos los fabricantes para cada motor.

La plataforma Java provee el administrador de protocolos JDBC (Java Database Connectivity) con ese propósito. La idea es que JDBC permita que los programas se conecten con cualquier base de datos de cualquier motor, y accedan a ella de manera transparente usando SQL desde el interior de un programa Java, sin preocuparse por los detalles, reglas y procedimientos propios de cada motor: JDBC toma el requerimiento del programador y lo transforma en una petición válida para el motor de que se trate, devolviendo a su vez la respuesta.

Tipicamente una base de datos será accedida desde un programa "stand alone" o "aplicación" Java. Un programa que acceda a una base de datos en Java, deberá incluir el paquete `java.sql` y también eventualmente `jakarta.sql`.

Existen tecnologías de más alto nivel como JPA o Hibernate que permiten trabajar con objetos Java en lugar de escribir SQL directamente. Sin embargo, JDBC continúa siendo la base del acceso a datos en Java y permite comprender cómo se establece una conexión, cómo se ejecutan sentencias SQL y cómo se administran las transacciones.

Para conectar con una base de datos, el programa deberá en algún momento especificar la dirección de la misma, lo cual se hace en forma similar a la especificación de una dirección URL. El formato general de una dirección para una base de datos es el siguiente:

protocolo:subprotocolo:parámetros

Donde:

protocolo: es JDBC

subprotocolo: Es el nombre del driver del fabricante a usar

parámetros: Son valores que dependen del subprotocolo

Por ejemplo,

`jdbc:sqlserver://localhost;databaseName=pd;encrypt=false`

Para poder conectarse con una base de datos, debe ser registrado el DriverManager que será el encargado de seleccionar el controlador y crear la conexión. Para ello, se usa el método

 UNIVERSIDAD Blas Pascal	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JDBC	VERSIÓN: 1.3 VIGENCIA: 11-05-2020

estático `forName()` de la clase `Class`. Si se indica a este método el nombre completo (incluyendo los packages a los cuales pertenece) de una clase, el mismo intenta ubicar, cargar, y linkear la clase o interfaz en tiempo de ejecución. Por lo tanto, la siguiente invocación a `forName()` ubica, carga y linkea en tiempo de ejecución la clase `SQLServerDriver` para gestionar una conexión vía JDBC con una base de datos.

`Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");`

Habiendo ya cargado el driver `JDBC:SQLSERVER`, el método **`getConnection()`** de la clase `DriverManager` permite crear una conexión con una base de datos. La conexión misma se representa en Java como un objeto de la clase **`Connection`**, retornado por `getConnection()` en la forma que sigue:

`Connection conn = DriverManager.getConnection("jdbc:sqlserver://localhost;databaseName=fdc;encrypt=false", "sa", "pyxis");`

A partir del objeto de la clase `Connection` creado antes, podemos comenzar a ejecutar sentencias con la base de datos.

Las operaciones SQL pueden disparar una excepción de la clase `SQLException`, por lo cual es conveniente incluir un bloque `try - catch` para capturar esa eventual excepción.

Lo siguiente a definir es el modo de confirmación automática (`autocommit mode`) que determinará si cada sentencia SQL se confirma automáticamente después de ejecutarse o si se requiere una confirmación explícita del programador.

`conn.setAutoCommit(true);`

- Descripción:
 - Este es el modo predeterminado para las conexiones JDBC.
 - Cada instrucción SQL que se ejecuta se confirma automáticamente después de su ejecución.
- Ventajas:
 - Es fácil de usar para operaciones SQL sencillas y rápidas.
 - No requiere una gestión explícita de transacciones por parte del programador.
- Desventajas:
 - No es adecuado para operaciones que deben ser atómicas (es decir, todas deben completarse correctamente o ninguna debe completarse).
 - No permite agrupar múltiples operaciones SQL en una única transacción.

`conn.setAutoCommit(false);`

- Descripción:
 - Desactiva el modo de confirmación automática.
 - Las transacciones deben confirmarse explícitamente utilizando **`conn.commit()`**.
 - En caso de error, las transacciones pueden deshacerse utilizando **`conn.rollback()`**.
- Ventajas:
 - Control total: Permite agrupar múltiples operaciones SQL en una única transacción, asegurando que todas se completen correctamente antes de confirmar.
 - Integridad transaccional: Garantiza que las operaciones dependientes se completen juntas, manteniendo la consistencia de los datos.

	INGENIERÍA EN INFORMÁTICA – PLAN 2014 PROGRAMACIÓN DISTRIBUIDA Y COMPONENTES – 9º CUATRIMESTRE	
	APUNTE DE JDBC	VERSIÓN: 1.3 VIGENCIA: 11-05-2020

- Recuperación de errores: En caso de error, se puede revertir la transacción para deshacer todos los cambios.
- **Desventajas:**
 - Complejidad adicional: Requiere una gestión explícita de las transacciones.
 - Posible impacto en el rendimiento si no se maneja correctamente, ya que las transacciones abiertas pueden mantener bloqueos en la base de datos.

Las clases que nos permiten construir la sentencia a ejecutar son:

Clase	Propósito
Statement	Permite definir sentencias que no requieren ser parametrizadas.
PreparedStatement	Permite definir sentencias parametrizadas.
CallableStatement	Permite definir sentencias de invocación a procedimientos almacenados.

Es importante considerar que los tipos de datos en SQL no se llaman igual que en Java. Aquí va una tabla de equivalencias de los tipos principales:

Tipo SQL	Tipo Java
BIGINT	long
INTEGER o INT	int
SMALLINT	short
TINYINT	short
FLOAT(n)	double
REAL	float
DOUBLE	double
DECIMAL o NUMERIC	java.math.BigDecimal
CHAR(n)	String
VARCHAR(n)	String
BIT	boolean
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp

Para ejecutar la sentencia SQL existen los siguientes métodos:

Método	Uso	Retorno
executeQuery()	SELECT	ResultSet
executeUpdate()	INSERT, UPDATE, DELETE	int
execute()	Procedimientos almacenados o sentencias generales	boolean